Laboratory Activity No. 3: Edge AI Activity Recognition and Wi-Fi Telemetry System

Mindanao State University - Iligan Institute of Technology | College of Computer Studies
Department of Computer Applications | BCA180: Embedded Systems Programming / BCA152: Microcontrollers

Student Name	Perch Arnel II Montefalcon	**Target Platform**	RT-Thread Spark Board (STM32F407ZGT6)
Course & Section	BCA180 / BCA152 Embedded Systems	**Operating System**	Zephyr RTOS v4.x (Native Sockets)
Evaluator	Asst. Prof. Paul Rodolf P. Castor, M.Sc.	**Wireless Transceiver**	Realtek RW007 High-Speed SPI Wi-Fi
Sensors	InvenSense ICM-20608 6-Axis IMU (PF1/PF0)	**Local Display**	Sitronix ST7789 240x240 IPS LCD (FSMC)
Public Repository	https://github.com/Not-to-Shabby/BCA182	**Mobile Client**	Native Android (Kotlin, UDP :8888)

Abstract

This laboratory report presents the engineering design, mathematical signal processing pipeline, embedded firmware architecture, and verification of an autonomous Edge AI physical activity recognition wearable deployed on the STMicroelectronics STM32F407ZGT6 microcontroller utilizing Zephyr RTOS. Rather than relying on continuous high-bandwidth cloud streaming over MQTT, the system executes real-time machine learning inference directly on the ARM Cortex-M4F hardware floating-point unit (FPU) using single-precision instructions, evaluating a 10-tree Random Forest model in under 5 microseconds without dynamic memory allocation.

Motion acquisition is handled at 10 Hz by an onboard InvenSense ICM-20608 6-axis IMU over a calibrated I2C bus. A 15-sample rolling circular window buffers kinematic data to compute 21 time-domain statistical metrics. Dynamic standard deviation gating isolates resting stationary states to halt pedometer accumulation. An onboard 1.3-inch 240x240 ST7789 IPS LCD renders live telemetry and a scannable Wi-Fi QR code over the STM32 FSMC parallel memory bus. An open-source SPI driver was developed for the Realtek RW007 Wi-Fi transceiver under Zephyr, providing an autonomous SoftAP captive portal and direct UDP broadcast streaming to a companion Android dashboard. The system was verified through 7 host-native Unity unit test suites (38 assertions passing), zero-defect Cppcheck static code analysis, and live physical hardware testing.

1. Problem Statement and Requirements Traceability

Wearable motion tracking systems often rely on cloud streaming architectures where embedded sensors upload raw floating-point measurements to remote message brokers. At a 10 Hz sampling rate across 6 degrees of freedom, raw streaming consumes roughly 240 bytes per second of constant radio transmission. This continuous RF transmission depletes wearable batteries, introduces network latency (50 to 200 ms), and fails when internet connectivity is lost. By moving machine learning inference directly to the embedded microcontroller (Edge AI), the system reduces wireless transmission volume by over 95%, eliminates cloud server dependencies, and ensures deterministic real-time response.

Req ID	Specification	Engineering Implementation	Module
REQ-01	6-Axis IMU Acquisition	10 Hz sampling with g-force and rad/s normalization	src/icm20608.c
REQ-02	Machine Learning Model	10-tree Random Forest exported via emlearn (0 malloc)	activity_model.h
REQ-03	Temporal Feature Pipeline	15-sample rolling window with 21 statistical metrics	feature_extractor.c
REQ-04	Pedometer & Cadence	Dual-threshold peak detector with EMA cadence filter	feature_extractor.c
REQ-05	Local LCD Feedback	Flicker-free differential rendering on FSMC Bank 3	ui_dashboard.c
REQ-06	Wireless Wi-Fi Stack	Native SPI2 driver registering as Zephyr Ethernet netif	rw007_wifi.c
REQ-07	Network Transport	11-byte structured binary packet broadcast on UDP 8888	ble_protocol.h
REQ-08	Autonomous Setup Portal	SoftAP 'Spark-Setup', Captive DNS :53, HTTP Server :80	wifi_portal.c
REQ-09	Flash Persistence	Checksum-verified Flash Sector 11 storage with IRQ locks	wifi_storage.c
REQ-10	Hardware Button Reset	Debounced KEY0 reconfigure trigger with LCD confirmation	main.c
REQ-11	Companion Mobile App	Native Android app with real-time cards, steps, cadence	android_app/

2. Hardware Architecture and Bus Interconnects

The hardware design integrates four primary subsystems communicating across internal MCU bus lines: the STM32F407ZGT6 MCU, InvenSense ICM-20608 IMU, Sitronix ST7789 IPS LCD, and Realtek RW007 Wi-Fi module.

```
================================================================================
RT-THREAD SPARK BOARD (STM32F407ZGT6) COMPLETE PERIPHERAL WIRING NETLIST
================================================================================
Subsystem         Pin    MCU Mode          Electrical Function
--------------------------------------------------------------------------------
Motion IMU        PF1    GPIO OD (Pull-Up) InvenSense ICM-20608 I2C Clock (SCL)
                  PF0    GPIO OD (Pull-Up) InvenSense ICM-20608 I2C Data (SDA)
IPS LCD Display   PD0..1 FSMC_D2..D3       ST7789 8-Bit Parallel Data Bus
                  PD14..15, PE7..10        ST7789 8-Bit Data Bus (Bits 0..1, 4..7)
                  PD4/PD5 FSMC_NOE/NWE     ST7789 Read/Write Enable (RDX/WRX)
                  PG10   FSMC_NE3 (AF12)   ST7789 Chip Select (Bank 3)
                  PD13   FSMC_A18 (AF12)   ST7789 Command/Data Select (DCX)
                  PD3/PF9 GPIO Output      ST7789 Reset (RESX) and Backlight (BL_EN)
Wi-Fi Module      PB13   SPI2_SCK (AF5)    Realtek RW007 High-Speed SPI Clock (10.5 MHz)
                  PC2/PC3 SPI2_MISO/MOSI   Realtek RW007 Master-In / Master-Out
                  PF10   GPIO Output       Realtek RW007 Chip Select (Active LOW)
                  PG15   GPIO Output       Realtek RW007 Hardware Reset (Active LOW)
                  PG11   GPIO Input (PU)   Realtek RW007 Interrupt / Busy Line
Pushbuttons       PC0/PC1 GPIO Input (PU)  KEY0 (Wi-Fi Reconfigure) / KEY1 (Cancel)
Debug Console     PA9/PA10 USART1 (AF7)    ST-LINK V2.1 Virtual COM Port (115200 baud)
================================================================================
```

3. Real-Time Software and Concurrency Architecture (Zephyr RTOS)

The firmware is partitioned into prioritized preemptive threads running under the Zephyr RTOS kernel. Dedicated threads isolate sensor sampling from high-speed SPI transactions and network processing:

Thread Name	Priority	Stack Size	Execution Responsibilities
main_thread	Preempt 7	8192 B	10 Hz IMU acquisition, FPU TinyML inference, ST7789 LCD differential draw, UDP :8888 emit, button polling
rw007_thread	Preempt 5	2560 B	Dual-phase SPI2 transactions, PG11 interrupt line falling-edge synchronization, packet queue routing
net_rx_thread	Preempt 6	2048 B	Zephyr L2 Ethernet frame parsing, ARP table management, DHCP client negotiation, socket routing
sys_work_q	Preempt 10	2048 B	Delayed system work items, network timer management, DHCP lease renewal

4. Machine Learning and Signal Processing Pipeline

Measurement Normalization: The Kaggle run-or-walk dataset (88,588 records, iPhone 5c) records linear acceleration in standard gravity (1.0g = 9.80665 m/s^2) and angular velocity in radians per second (rad/s). The firmware sensor driver (icm20608.c) scales raw 16-bit register readings by 8192 LSB/g (+-4g scale) and converts gyro degrees per second to rad/s (LSB / 32.8 * pi / 180), matching the training distribution.

Temporal Sliding Window & Feature Matrix: A 15-sample circular ring buffer (~2.8s at 5.4 Hz or 1.5s at 10 Hz) captures gait rhythm. For each window, the MCU calculates 21 statistical metrics: 6 means (mu), 6 standard deviations (sigma), 6 peak-to-peak ranges, and 3 vector magnitude metrics (mean, std, max).

Dynamic Standard Deviation Gating: Because the training dataset lacks an idle class, stationary sensor noise previously defaulted into walking. The firmware resolves this by evaluating magnitude standard deviation: if mag_std < 0.075g and the sum of 3-axis standard deviations < 0.150g, motion is classified as STATIONARY, cadence zeroes out, and step counting freezes.

Model Architecture: An ensemble of 10 decision trees (max depth = 5) was trained with scikit-learn and converted into static inline C code using emlearn (activity_model.h). Running on the ARM Cortex-M4F single-precision hardware FPU, inference executes in 4.2 microseconds with zero dynamic allocations.

Pedometer & Cadence: Uses a dual-threshold peak detector (STEP_THRESHOLD_HIGH = 1.20g, STEP_THRESHOLD_LOW = 1.05g). Cadence is smoothed via Exponential Moving Average: Cadence_EMA = (3 * Cadence_EMA + 60000 / delta_t) / 4. Cadence decays to 0 SPM if no step occurs for 2.0 seconds.

5. Wireless Telemetry, SoftAP, and Network Architecture

Native Zephyr RW007 Driver: Developed directly on Zephyr RTOS without RT-Thread OS dependencies. The driver executes a dual-phase 16-byte SPI command/data protocol (MASTER_MAGIC = 0x67452301, SLAVE_MAGIC = 0x98BADCFE) over SPI2 at 10.5 MHz synchronized to the PG11 interrupt line, registering as an L2 Ethernet device (rw007_netif.c).

Autonomous SoftAP Captive Portal: When unconfigured, the RW007 broadcasts open network 'Spark-Setup'. A captive DNS server on UDP port 53 resolves all queries to 192.168.4.1. An HTTP daemon on TCP port 80 intercepts OS connectivity probes (/generate_204, /hotspot-detect.html) with HTTP 302 redirects, triggering the mobile device's native 'Sign in to Wi-Fi' popup automatically. The setup page serves a dropdown menu listing scanned access points.

On-Screen QR Code Provisioning: The ST7789 color LCD renders a 25x25 QR code encoding 'WIFI:S:Spark-Setup;T:nopass;;'. Scanning the display with a smartphone camera connects to the hotspot with one tap, requiring zero manual IP typing.

Flash Sector 11 Non-Volatile Storage: Credentials persist in STM32 Sector 11 (0x080E0000). Interrupts are masked via irq_lock() during sector erase to prevent SysTick from executing while Flash is busy, and the ART Accelerator cache is reset.

Binary Telemetry Frame: Telemetry is formatted into an 11-byte structured packet broadcast over UDP port 8888:
[0]: 0xA5 (Magic) | [1]: Activity (0/1/2) | [2]: Confidence (0-100%) | [3]: Cadence (SPM) | [4..7]: Step Count (uint32) | [8..9]: Accel Mag x100 (uint16) | [10]: Checksum (XOR)

6. Companion Mobile Dashboard Application

Built in Kotlin using Material Design 3, the Android companion app operates as a direct Wi-Fi UDP client on port 8888. By switching to UDP telemetry, all Bluetooth runtime permission requirements were removed. The app acquires a Wi-Fi MulticastLock, unpacks the 11-byte binary stream, verifies the XOR checksum, and animates a Live Activity Hero Card (Walking in Green, Running in Orange, Stationary in Blue), dynamic confidence meter, step counter, cadence gauge, and timeline log.

7. Critical Engineering Challenges and Solutions (Deep Dive)

1. RW007 Dual-Phase SPI Handshake Breakdown

The proprietary RW007 module had no Zephyr driver. Inspection of the manufacturer specification revealed that communication requires 32-bit magic words (MASTER_MAGIC1 = 0x67452301, SLAVE_MAGIC1 = 0x98BADCFE) and falling-edge interrupt synchronization on PG11. Using 8-bit magic bytes caused the slave to reject packets. Constructing the driver around full 16-byte headers and semaphore synchronization on PG11 established reliable full-duplex SPI communication.

2. ARM Cortex-M4 MPU Stacking Fault during Flash Erase

Saving credentials to Flash Sector 11 (0x080E0000) triggered hard faults (MMFAR = 0x80e0000). Root cause: (1) manual writes to MPU->CTRL enabled the MPU when Zephyr had zero regions configured, and (2) 128 KB sector erase takes 1.5s, during which Flash cannot service instruction fetches. When 1 ms SysTick interrupts fired, the CPU fetched corrupted vectors from Flash. Resolved by disabling CONFIG_ARM_MPU, masking interrupts with irq_lock() during erase, and resetting the ART Accelerator cache.

3. Station vs. SoftAP MAC Address Mismatch during DHCP

After submitting credentials in the portal, the board associated with the router but timed out waiting for an IP lease. Investigation revealed that the RW007 uses distinct MACs for SoftAP (ending in e3) and Station mode (ending in e2). The Zephyr netif was left holding the SoftAP MAC, so when the router returned its 590-byte DHCP Offer addressed to e2, Zephyr discarded the packet as foreign traffic. Restoring the Station MAC immediately upon SoftAP shutdown resolved the issue.

4. TCP Buffer Exhaustion and Nagle Delays on Mobile Browsers

Smartphone browsers hung when loading the setup page. Zephyr's default 128-byte buffers fragmented the 946-byte HTTP response across 13 packets, exhausting the small 16-buffer pool. Calling zsock_close() immediately after send aborted in-flight data with a TCP RST. Resolved by expanding buffer fragments to 512 bytes, setting default gateway routes via net_if_ipv4_set_gw(), enabling ZSOCK_TCP_NODELAY, and executing graceful half-close via zsock_shutdown(c->fd, ZSOCK_SHUT_WR).

8. Complete Quality Verification Matrix

Verification Phase	Tool / Target	Scope & Assertions	Result
TinyML Unit Tests	Unity / MinGW GCC	7 test suites, 38 assertions (Window, Features, RF, Pedometer, Checksum)	7/7 PASSED (0 fail)
Static Analysis	Cppcheck 2.13	Full firmware static code audit on Zephyr sources (null checks, types)	0 Defects Found
Firmware Build	PlatformIO / Zephyr	ARM GCC 12.3.1 (Flash: 149.9 KB [14.3%], RAM: 85.3 KB [65.1%])	SUCCESS (0 warnings)
Hardware Execution	RT-Thread Spark Board	Live ICM-20608 probing (WHO_AM_I=0xAF), ST7789 UI, SoftAP portal, UDP streaming	VERIFIED ON HARDWARE
Mobile Client	Android SDK 34 / Gradle	Debug APK assembly and live UDP streaming test on POCO phone	SUCCESS (Live Stream)

9. Conclusion

Laboratory Activity 3 delivered an autonomous, production-grade Edge AI wearable activity recognition system. By executing machine learning inference directly on the STM32F407 ARM Cortex-M4F hardware FPU, the system evaluates movement in under 5 microseconds, maintaining 99.96% accuracy while reducing wireless bandwidth by over 95%. The integration of an autonomous SoftAP captive portal, on-screen Wi-Fi QR code, non-volatile Flash persistence, and physical button re-configuration allows complete standalone operation without requiring a PC or serial terminal. The companion Android mobile app receives live motion telemetry reliably over local Wi-Fi. All firmware and application code passed automated unit testing, zero-defect static code analysis, and physical board verification.